

GRPO Explained

Table of Contents

- Why Do We Need RL?
- What Problem Does GRPO Solve?
- GRPO Loss
 - logprobs
 - Surrogate Function
 - KL Penalty
- GRPO Algorithm (Pseudocode)

Why Do We Need RL?

Why is SFT not enough? Why do we put RL on top?

- SFT teaches the LLM to reproduce the behavior that's currently in the dataset.
- With RL, the model is encouraged to discover.

Example: User asks, "What's the weather tomorrow?"

SFT is like watching a driving instructor. RL is like getting behind the wheel, driving and learning.

Why do we need (usually) SFT before RL?

No SFT → 1000 attempts → 0 correct

SFT → 1000 attempts → 150 correct Uygur Kurt

What Problem Does GRPO Solve?

RLHF: Compare the rollout against a learned expectation from the critic model.

GRPO: Compare the rollout against the average of the rollouts generated for the same prompt (which becomes a group).

The critic model is expensive. With GRPO, we don't need it.

We can think of GRPO and RLHF as a student taking an exam.

GRPO LOSS

$$L(\theta) = \frac{1}{N} \sum_{i=1}^G \sum_{t=1}^{T_i} [-S_{i,t}(\theta) + \beta D_{i,t}(\theta)]$$

Handwritten annotations: $\checkmark$ above S , $\checkmark$ above P , and "KL penalty" with an arrow pointing to $D_{i,t}(\theta)$.

Where $\hookrightarrow$ iterate Group $\hookrightarrow$ iterate each rollout

- G is the number of groups
- T_i is the i 'th generated rollout within the group G .
- t is the t 'th token within T_i
- N is the total number of valid generated tokens within the current group.

GRPO Loss (logprobs) $b^{\log_b(x)} = x$

$$\ell_{i,t}^{\theta} = \ln \pi_{\theta}(y_{i,t} | \overbrace{x, y_{i,<t}}^{\text{prompt}}) \quad (\text{new_logprobs})$$

$$\ell_{i,t}^{\text{old}} = \ln \pi_{\text{old}}(y_{i,t} | x, y_{i,<t}) \quad (\text{old_logprobs})$$

$$\ell_{i,t}^{\text{ref}} = \ln \pi_{\text{ref}}(y_{i,t} | x, y_{i,<t}) \quad (\text{ref_logprobs})$$

Where

- i is the index of the current rollout
- t is the current token in the current rollout
- x is the prompt

Where will we use these **log** probabilities?

Why do we use **log**?

$$\frac{\text{new_probs}}{\text{old_probs}} = e^{\ln(\frac{\text{new_probs}}{\text{old_probs}})} = \exp(\ln(\frac{\text{new_probs}}{\text{old_probs}}))$$

numerically unstable.

$$\frac{1.1 \times 10^{-20}}{10^{-20}} = 1.1 \quad \ln(1.1 \times 10^{-20}) = -45.96$$

$$\ln(10^{-20}) = -46.05$$

$$\exp(-45.96 - (-46.05))$$

$$\frac{0}{0} = \text{NaN}$$

$$= \exp(\log) \approx 1.1$$

GRPO Loss (Surrogate Function)

Since we're trying to minimize $\underline{L(\theta)}$, we have to maximize clipped surrogate $S_{i,t}$.

$$S_{i,t}(\theta) = \min[\underbrace{r_{i,t}(\theta)}_{\leftarrow \hat{A}_i}, \underbrace{\text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)}_{\leftarrow \hat{A}_i}] \hat{A}_i$$

Where the probability ratio $r_{i,t}$ is $r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$, which equals, and is calculated with $r_{i,t}(\theta) = \exp\left(\underbrace{\ell_{i,t}^{\theta}}_{\leftarrow \text{new log probs}} - \underbrace{\ell_{i,t}^{\text{old}}}_{\leftarrow \text{old log probs}}\right)$ while implementing for numerical stability.

GRPO Loss (Surrogate Function)

And where advantage $\hat{A}_i$ is $\hat{A}_i = \frac{R_i - \bar{R}}{\sigma_R + \delta}$

Where

- R_i is the reward of rollout i
- $\bar{R}$ is the average reward of the group
- σ_R is the standard deviation of the group rewards (we divide by it to normalize the scale of the advantages)
- δ is a small constant to prevent division by 0

$$\begin{array}{rcl} 4 & \rightarrow & 1.5 \uparrow \\ \hline 3 & \rightarrow & 0.5 \\ 2 & \rightarrow & -0.5 \\ \hline 1 & \rightarrow & -1.5 \downarrow \\ \hline R & & A \end{array}$$

GRPO Loss (Surrogate Function)

The advantage tells us how well this rollout performed compared with the other rollouts in the same group.

- A positive advantage $\hat{A}$ reinforces that rollout behavior
- A negative advantage $\hat{A}$ discourages that rollout behavior

The larger the magnitude of the advantage, the stronger the learning signal.

Why $\hat{A}$ behaves this way? Let's put things together (next slide).

GRPO Loss (Surrogate Function)

$$\uparrow S_{i,t}(\theta) = \min[r_{i,t}(\theta)\hat{A}_i, \underbrace{\text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_i}] \downarrow$$

$$\downarrow \uparrow r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t}) \uparrow \downarrow}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$$

GRPO Loss (KL Penalty)

Regular KL divergence would require us to go over the full vocabulary with:

$$D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) = \sum_{v \in \mathcal{V}} \pi_{\theta}(v | s) \log \frac{\pi_{\theta}(v | s)}{\pi_{\text{ref}}(v | s)}.$$

This requires a lot of calculations. However,

we have the sampling $y \sim \pi_{\theta}(\cdot | s)$. So, instead, we can do a much cheaper estimate.

For GRPO, Schulman's k3 KL estimator is used. Which is the following:

$$D_{i,t}(\theta) = \exp \left(\ell_{i,t}^{\text{ref}} - \ell_{i,t}^{\theta} \right) - \left(\ell_{i,t}^{\text{ref}} - \ell_{i,t}^{\theta} \right) - 1$$

$$1 - 0 - 1 = 0$$

This value is always positive.

GRPO Loss (Let's Put Things Together)

$$\underline{\underline{L(\theta)}} = \frac{1}{\underline{\underline{N}}} \sum_{i=1}^G \sum_{t=1}^{T_i} [-\underbrace{S_{i,t}(\theta)} + \underbrace{\beta D_{i,t}(\theta)}]$$

GRPO Algorithm (Pseudocode)

```
reference_policy = frozen_copy(policy)  
for prompt in dataset :  
    rollouts = generate_rollouts(policy, prompt, group_size)  
    old_logprobs = logprobs(policy, prompt, rollouts)  
    reference_logprobs = logprobs(reference_policy, prompt, rollouts)  
    rewards = reward(rollouts)  
    advantages = normalize_within_group(rewards)  
  
    for _ in ppo_epochs :  
        new_logprobs = logprobs(policy, prompt, rollouts)  
        ratio = exp(new_logprobs - old_logprobs)  
        surrogate = min(ratio · advantages, clip(ratio, 1 - clip_epsilon, 1 + clip_epsilon) · advantages)  
        kl = exp(reference_logprobs - new_logprobs) - (reference_logprobs - new_logprobs) - 1  
        loss = mean(-surrogate + kl_coefficient · kl)  
        update_policy(loss)
```